Python Grammar

Complete Beginner's Course

Learn Python from Scratch • No Experience Required

50 Interactive Slides

INTRODUCTION

What is Python?

- Python is a high-level, interpreted programming language created by Guido van Rossum in 1991.
- It emphasizes code readability and simplicity — you can do more with fewer lines than Java or C++.
- Python is used in web development, data science, AI, automation, scripting, and more.
- It's consistently ranked as the most popular programming language in the world.

```
# Your very first Python program!
print("Hello, World!")

# Output:
# Hello, World!
```

Python: 'Batteries included' — powerful out of the box!

INTRODUCTION

Why Learn Python?

Simple Syntax

Reads like English — easy to learn and write.

Versatile

Web apps, AI, data science, automation.

Huge Community

Millions of devs, libraries, tutorials.

Job Market

#1 most demanded language for data & AI roles.

Open Source

Free to use and distribute forever.

Cross-Platform

Runs on Windows, Mac, Linux.

SETUP

Installing Python

- Visit python.org and download the latest version (3.x recommended).
- Windows: Run installer and check 'Add Python to PATH'.
- Mac: Use Homebrew → brew install python3
- Linux: Usually pre-installed. Check with: python3 --version

```
# Check Python version in terminal:
python --version
# Output: Python 3.11.x

# Open the interactive shell:
python3
>>> print("Python is ready!")
Python is ready!
>>> exit()
```

Tip: Use VS Code or PyCharm as your editor!

PHILOSOPHY

The Zen of Python

```
import this

# Beautiful is better than ugly.
# Explicit is better than implicit.
# Simple is better than complex.
# Complex is better than complicated.
# Readability counts.
# Errors should never pass silently.
# There should be one obvious way to do it.
# Now is better than never.
```

- These are the guiding principles of Python design.
- Run 'import this' in Python to see the full list.
- These ideas explain WHY Python code looks so clean.
- Python encourages writing code that humans can read.

FUNDAMENTALS

Variables

- Variables store data in memory. Python creates them automatically when you assign a value.
- No need to declare the type(loosely typed) — Python figures it out!
- Use meaningful names: age, user_name, total_score.
- Variables are case-sensitive: age ≠ Age ≠ AGE.

```
name = "Alice"          # string
age = 25                # integer
height = 5.7            # float
is_student = True       # boolean

print(name)    # Alice
print(age)     # 25
print(type(age))  # <class 'int'>
```

Variable Anatomy

```
name  =  'Alice'
 ↑        ↑
identifier  value
```

Naming Rules

```
✓ my_var, count, total
✗ 2fast, my-var, class
```

FUNDAMENTALS

Core Data Types

int

Whole numbers

```
age = 25
count = -10
```

float

Decimal numbers

```
pi = 3.14
temp = -0.5
```

str

Text / String

```
name = "Alice"
word = 'hi'
```

bool

True or False

```
active = True
done = False
```

DATA TYPES

Strings

```
name = "Python"
greeting = 'Hello'

# Concatenation
full = greeting + ", " + name   # Hello, Python

# Length
print(len(name))   # 6

# Upper / Lower
print(name.upper())   # PYTHON
print(name.lower())   # python

# f-strings (modern way!)
age = 30
print(f"My age is {age}")   # My age is 30
```

- Strings are text enclosed in quotes (single or double).
- Strings are immutable — you can't change a character directly.
- Use f-strings for clean string formatting.
- Strings support indexing: name[0] → 'P'
- Negative indexing: name[-1] → 'n'

Useful String Methods

```
s = "  Hello, Python World!  "

print(s.strip())         # Remove whitespace
print(s.replace("Python", "Beautiful"))
print(s.split(","))      # ['  Hello', ' Python...']
print(s.startswith("  Hello"))  # True
print(s.endswith("!  "))        # True
print(s.find("Python"))  # 9 (index)
print(s.count("l"))      # 3
print("42".isnumeric())  # True
print("hello".capitalize())  # Hello
print("hi".center(10, "*"))  # ****hi****
```

- .strip() — removes whitespace from both ends
- .replace(a, b) — replaces all occurrences of a with b
- .split(sep) — splits into a list
- .find(sub) — returns index of first match
- .upper() / .lower() — change case
- .format() — template formatting

FUNDAMENTALS

Print & User Input

```
# Basic printing
print("Hello!")             # Hello!
print("Name:", "Alice")     # Name: Alice

# Multiple values with sep
print("A", "B", "C", sep="-")  # A-B-C

# No newline at end
print("Enter name: ", end="")

# f-string formatting
name = "Bob"
age = 20
print(f"{name} is {age} years old")

# User input (always returns string!)
name = input("What is your name? ")
age = int(input("Your age? "))   # convert to int
print(f"Hi {name}, you are {age}!")
```

- print() outputs text to the screen.
- Use sep= to set separator between values.
- Use end= to control what comes after the print.
- input() always returns a STRING — convert it!
- int(input()) converts user input to integer.
- float(input()) converts to decimal number.

OPERATORS

Arithmetic Operators

```
a = 10
b = 3

print(a + b)   # 13  — Addition
print(a - b)   # 7   — Subtraction
print(a * b)   # 30  — Multiplication
print(a / b)   # 3.333 — True Division
print(a // b)  # 3   — Floor Division
print(a % b)   # 1   — Modulus (remainder)
print(a ** b)  # 1000 — Exponentiation

# Real example: km to miles
km = 100
miles = km * 0.621371
print(f"{km} km = {miles:.2f} miles")
```

Operator	Symbol	Example
Addition	+	5 + 3 = 8
Subtraction	-	5 - 3 = 2
Multiply	*	5 * 3 = 15
Divide	/	5 / 2 = 2.5
Floor Div	//	5 // 2 = 2
Modulus	%	5 % 2 = 1
Power	**	2 ** 8 = 256

Assignment Operators

```
x = 10        # Basic assignment

x += 5        # x = x + 5  → 15
x -= 3        # x = x - 3  → 12
x *= 2        # x = x * 2  → 24
x /= 4        # x = x / 4  → 6.0
x //= 2       # x = x // 2 → 3.0
x **= 3       # x = x ** 3 → 27.0
x %= 5        # x = x % 5  → 2.0

# String multiplication!
line = "-" * 20
print(line)   # --------------------

word = "Ha"
print(word * 3)  # HaHaHa
```

- Assignment operators are shortcuts for updating variables.
- x += 5 is the same as x = x + 5.
- Python does NOT have ++ or -- operators like JavaScript/C++.
- Python also has no === (strict equality) — use == instead.
- String * number repeats the string!

OPERATORS

Comparison & Logical Operators

```
a, b = 10, 5

# Comparison (returns True or False)
print(a == b)   # False — equal
print(a != b)   # True  — not equal
print(a > b)    # True  — greater than
print(a < b)    # False — less than
print(a >= 10)  # True  — greater or equal
print(a <= 9)   # False — less or equal

# Logical operators
print(a > 5 and b < 10)  # True (both true)
print(a > 15 or b < 10)  # True (one is true)
print(not a > 5)         # False (flipped)
```

- Comparison operators return Boolean: True or False.
- and: both sides must be True.
- or: at least one side must be True.
- not: flips True to False and vice versa.
- Used heavily in if-statements and while loops.

OPERATORS

Identity & Membership Operators

```
# Identity operators: compare memory location
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is c)     # True  — same object!
print(a is b)     # False — different objects
print(a is not b) # True

# Membership operators: check if in collection
fruits = ["apple", "banana", "mango"]

print("apple" in fruits)      # True
print("grape" in fruits)      # False
print("grape" not in fruits)  # True

word = "Python"
print("Py" in word)           # True
```

- is checks if two variables point to the same object in memory.
- == checks if values are equal (different from is!).
- in checks if a value exists inside a sequence.
- not in checks if a value does NOT exist.
- These are very useful in loops and conditionals.

CONTROL FLOW

If Statements

```
age = 18

if age >= 18:
    print("You are an adult!")

# if-else
if age >= 18:
    print("You can vote.")
else:
    print("Too young to vote.")

# if-elif-else
score = 75
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
# Output: Grade: C
```

- Python uses INDENTATION (4 spaces) to define blocks — no curly braces!
- if runs code when condition is True.
- elif = else if — checks another condition.
- else runs when all conditions are False.
- You can chain as many elif blocks as needed.

Combining Conditions

```
# Using 'and'
age = 20
has_id = True

if age >= 18 and has_id:
    print("Entry allowed!")

# Using 'or'
day = "Saturday"
if day == "Saturday" or day == "Sunday":
    print("It's the weekend!")

# Using 'not'
is_raining = False
if not is_raining:
    print("Let's go outside!")

# Nested if
num = 15
if num > 0:
    if num % 2 == 0:
        print("Positive even")
```

- Combine multiple conditions with and, or, not.
- and: ALL conditions must be True.
- or: at least ONE must be True.
- not: reverses the condition.
- You can nest if-statements inside each other.

For Loops

```
# Loop over a list
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
# apple, banana, cherry

# Loop with range()
for i in range(5):
    print(i)   # 0 1 2 3 4

for i in range(1, 6):
    print(i)   # 1 2 3 4 5

for i in range(0, 10, 2):
    print(i)   # 0 2 4 6 8

# Loop over string
for char in "Python":
    print(char)   # P y t h o n
```

- for loops iterate over any sequence: list, string, range.
- range(n) generates 0 to n-1.
- range(start, stop) — from start up to (not including) stop.
- range(start, stop, step) — with custom step.
- The loop variable (fruit, i, char) gets each value.

While Loops

```
# Basic while loop
count = 0
while count < 5:
    print(count)   # 0 1 2 3 4
    count += 1

# While with user input
password = ""
while password != "secret123":
    password = input("Enter password: ")
print("Access granted!")

# Loop with break
n = 0
while True:
    print(n)
    n += 1
    if n >= 3:
        break    # exit the loop
# 0, 1, 2
```

```
# Loop with continue (skip even numbers)
```

- while loops run as long as the condition is True.
- Be careful of infinite loops — always update the condition!
- break exits the loop immediately.
- continue skips to the next iteration.
- while True: creates an infinite loop (use break to exit).

LOOPS

Iterators & Reverse Iteration

```
# iter() and next()
numbers = [10, 20, 30]
it = iter(numbers)

print(next(it))   # 10
print(next(it))   # 20
print(next(it))   # 30

# Iterating in reverse
fruits = ["apple", "banana", "cherry"]
for fruit in reversed(fruits):
    print(fruit)
# cherry, banana, apple

# Enumerate — get index + value
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
# 0: apple
# 1: banana
# 2: cherry
```

```
# Looping through nested lists
```

- iter() creates an iterator from a sequence.
- next() retrieves the next item.
- reversed() iterates in reverse order.
- enumerate() gives index + value pairs.
- Nested loops for 2D structures (matrices, grids).

Defining Functions

```
# Basic function
def greet():
    print("Hello, World!")

greet()   # Hello, World!

# Function with parameters
def greet_user(name):
    print(f"Hello, {name}!")

greet_user("Alice")   # Hello, Alice!

# Function with return value
def add(a, b):
    return a + b

result = add(5, 3)
print(result)   # 8

# Function with multiple returns
def min_max(numbers):
    return min(numbers), max(numbers)
```

- def keyword defines a function.
- Functions group reusable code into named blocks.
- Parameters are inputs; arguments are the values passed in.
- return sends a value back to the caller.
- A function can return multiple values as a tuple.

Function Arguments

```
# Default arguments
def greet(name, msg="Hello"):
    print(f"{msg}, {name}!")

greet("Alice")           # Hello, Alice!
greet("Bob", "Howdy")   # Howdy, Bob!

# Keyword arguments
def order(item, qty=1, size="medium"):
    print(f"{qty} {size} {item}(s)")

order("coffee", size="large", qty=2)
# 2 large coffee(s)

# Arbitrary arguments (*args)
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3, 4))   # 10

# Arbitrary keyword arguments (**kwargs)
def info(**details):
```

- Default args have a fallback value if not provided.
- Keyword args let you pass values by name, in any order.
- *args collects unlimited positional arguments into a tuple.
- **kwargs collects unlimited keyword arguments into a dict.
- Mix them: positional first, then keyword, then *args, then **kwargs.

Built-in Functions

```
# Common built-ins
print(len("Python"))      # 6
print(abs(-42))           # 42
print(round(3.7))         # 4
print(min(5, 2, 9))       # 2
print(max(5, 2, 9))       # 9
print(sum([1, 2, 3, 4]))  # 10
print(type(3.14))         # <class 'float'>

# Type conversion
print(int("42"))          # 42
print(float("3.14"))      # 3.14
print(str(100))           # '100'
print(bool(0))            # False
print(bool("hi"))         # True

# Evaluation
print(eval("2 + 3 * 4"))  # 14
print(chr(65))            # A
print(hex(255))           # 0xff
print(id(42))             # memory address
```

- Python ships with dozens of built-in functions — no import needed.
- int(), float(), str(), bool() convert between types.
- len() works on strings, lists, tuples, dicts, sets.
- abs() returns absolute value.
- eval() evaluates a string as Python expression.
- Don't overwrite built-ins! (e.g., don't name a variable 'list').

Recursive Functions

```
# Factorial using recursion
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))   # 120
# 5 * 4 * 3 * 2 * 1 = 120

# Fibonacci using recursion
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

for i in range(8):
    print(fib(i), end=" ")
# 0 1 1 2 3 5 8 13

# Countdown example
def countdown(n):
    if n <= 0:
```

- A recursive function calls itself.
- Must have a BASE CASE to stop recursion.
- Without base case → infinite recursion → crash!
- Stack memory stores each function call.
- Python's default recursion limit is 1000.

Lists

```
# Create a list
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", True, 3.14]

# Access by index (starts at 0)
print(fruits[0])    # apple
print(fruits[-1])   # cherry (last item)

# Slicing
print(numbers[1:4])   # [2, 3, 4]
print(numbers[:3])    # [1, 2, 3]
print(numbers[::2])   # [1, 3, 5] (every 2nd)

# Modifying lists
fruits[1] = "mango"
fruits.append("grape")    # add to end
fruits.insert(0, "kiwi") # insert at index 0
print(fruits)
# ['kiwi', 'apple', 'mango', 'cherry', 'grape']
```

- Lists are ordered, mutable (changeable) collections.
- Can hold ANY mix of data types.
- Index starts at 0. Negative index counts from the end.
- Slicing: list[start:stop:step].
- Lists are very commonly used in Python programs!

DATA STRUCTURES

List Methods

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]

nums.append(7)          # Add 7 to end
nums.extend([8, 9])     # Add multiple items
nums.insert(0, 0)       # Insert 0 at index 0
nums.remove(1)          # Remove first '1'
popped = nums.pop()     # Remove & return last
nums.sort()             # Sort ascending
nums.sort(reverse=True) # Sort descending
nums.reverse()          # Reverse in place
nums.clear()            # Remove everything

fruits = ["a", "b", "a", "c"]
print(fruits.count("a"))  # 2
print(fruits.index("b"))  # 1
print(len(fruits))        # 4

# Combine lists
list1 = [1, 2]
list2 = [3, 4]
print(list1 + list2)     # [1, 2, 3, 4]
```

- .append() — adds one item to end.
- .extend() — adds all items from another iterable.
- .pop() — removes and returns last item (or by index).
- .sort() — sorts in place (modifies original list).
- .count() — counts occurrences of a value.
- .index() — finds the position of a value.

Tuples

```
# Create tuples
coords = (10, 20)
colors = ("red", "green", "blue")
single = (42,)   # Note the comma!

# Access by index
print(coords[0])   # 10
print(colors[-1])  # blue

# Tuples are IMMUTABLE — you can't change them!
# coords[0] = 99   ← This will raise TypeError!

# Unpack tuples
x, y = coords
print(x, y)   # 10 20

# Nested tuples
point_3d = (1, 2, (3, 4))
print(point_3d[2][0])   # 3

# Convert list ↔ tuple
my_list = [1, 2, 3]
```

- Tuples are ordered, IMMUTABLE sequences.
- Use () parentheses (or no brackets at all).
- Single-item tuple needs a trailing comma: (42,)
- Faster than lists due to immutability.
- Great for data that shouldn't change: coordinates, RGB colors.
- Unpacking: a, b = (1, 2) assigns values to variables.

Dictionaries

```
# Create a dictionary
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

# Access values
print(person["name"])         # Alice
print(person.get("age"))      # 30
print(person.get("email", "N/A"))  # N/A (safe!)

# Modify
person["age"] = 31
person["email"] = "alice@example.com"
del person["city"]

# Iterate
for key, value in person.items():
    print(f"{key}: {value}")

# Keys, values, items
```

- Dictionaries store key-value pairs (like a real dictionary).
- Keys must be unique and immutable (strings, numbers, tuples).
- Use .get() for safe access — returns None if key doesn't exist.
- del removes a key-value pair.
- .items() returns (key, value) pairs for iteration.
- Dicts are ordered (Python 3.7+).

Dictionary Methods

```
d = {"a": 1, "b": 2, "c": 3}

d.update({"d": 4, "a": 99})   # add/update
print(d)  # {'a': 99, 'b': 2, 'c': 3, 'd': 4}

popped = d.pop("b")    # remove and return value
print(popped)          # 2

d.setdefault("e", 5)   # set if key not present
print(d.get("e"))      # 5

print(d.keys())        # all keys
print(d.values())      # all values
print(d.items())       # all (key, value) pairs

copy_d = d.copy()      # shallow copy
d.clear()              # empty the dict

# Create from keys
new_d = dict.fromkeys(["x","y","z"], 0)
print(new_d)  # {'x': 0, 'y': 0, 'z': 0}
```

- .update() merges another dict in (overwrites duplicates).
- .pop(key) removes and returns the value.
- .setdefault() sets a value only if the key doesn't exist.
- .fromkeys(list, val) creates dict with default values.
- | operator merges two dicts (Python 3.9+).

Sets

```
# Create a set (unique, unordered)
fruits = {"apple", "banana", "cherry", "apple"}
print(fruits)   # {'cherry', 'apple', 'banana'}
# Note: 'apple' appears only once!

fruits.add("mango")
fruits.discard("banana")   # safe remove

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a & b)            # {3, 4} — intersection
print(a | b)            # {1,2,3,4,5,6} — union
print(a - b)            # {1, 2} — difference
print(a ^ b)            # {1,2,5,6} — symmetric diff

# Check subsets
print({1,2}.issubset(a))     # True
print(a.isdisjoint({7,8}))   # True (no overlap)

# frozenset — immutable set
```

- Sets store UNIQUE, UNORDERED values — no duplicates!
- Create with {} or set() — but set() for empty sets (not {}).
- & = intersection, | = union, - = difference.
- Fast membership testing: 'apple' in fruits is O(1).
- frozenset is an immutable version of set.

Slicing Sequences

```
s = "Hello, Python!"

# Basic slicing: [start:stop]
print(s[0:5])    # Hello
print(s[7:])     # Python!
print(s[:5])     # Hello
print(s[-7:])    # Python!

# Slicing with step: [start:stop:step]
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[::2])    # [0,2,4,6,8] every 2nd
print(nums[1::2])   # [1,3,5,7,9] odd items
print(nums[::-1])   # [9,8,7,...,0] reversed!

# Replace with slicing
nums[2:5] = [20, 30, 40]
print(nums)   # [0,1,20,30,40,5,6,7,8,9]

# Using slice object
my_slice = slice(1, 8, 2)
print(nums[my_slice])
```

- Syntax: sequence[start:stop:step]
- start: inclusive (default 0).
- stop: exclusive (default end).
- step: how many to skip (default 1).
- [::-1] is the classic way to reverse any sequence!
- Works on strings, lists, tuples.

MEMORY

Variables & Memory

```
# Check memory address with id()
x = 42
print(id(x))    # e.g. 140234567890

# Assign by value (for immutables)
a = 10
b = a
b = 20
print(a)   # 10 — unchanged!

# Assign by reference (for mutables like lists)
list1 = [1, 2, 3]
list2 = list1     # both point to same list!
list2.append(4)
print(list1)   # [1, 2, 3, 4] — changed!

# Shallow copy vs deep copy
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)     # 1 level
deep = copy.deepcopy(original)    # full copy
```

- Immutables (int, str, tuple) are assigned by value.
- Mutables (list, dict, set) are assigned by reference.
- copy.copy() makes a shallow copy — nested objects still shared.
- copy.deepcopy() makes a fully independent copy.
- id() shows the memory address of an object.

OOP

Classes & Object-Oriented Programming

```
class Dog:
    # Constructor
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    # Instance method
    def bark(self):
        return f"{self.name} says: Woof!"

    def info(self):
        return f"{self.name} is a {self.breed}"

# Create instances (objects)
dog1 = Dog("Rex", "Labrador")
dog2 = Dog("Buddy", "Poodle")

print(dog1.bark())    # Rex says: Woof!
print(dog2.info())    # Buddy is a Poodle

# Access attributes
print(dog1.name)      # Rex
```

- class defines a blueprint for creating objects.
- __init__ is the constructor — runs when object is created.
- self refers to the instance itself.
- Instance attributes: set with self.name = value.
- Instance methods: functions defined inside the class.
- Objects (instances) are created by calling: Dog('Rex', 'Lab').

OOP

Class Methods & Static Methods

```
class Counter:
    count = 0   # Class attribute (shared)

    def __init__(self):
        Counter.count += 1
        self.id = Counter.count

    @classmethod
    def get_count(cls):
        return cls.count

    @staticmethod
    def description():
        return "I count instances of Counter"

c1 = Counter()
c2 = Counter()
c3 = Counter()

print(Counter.get_count())    # 3
print(Counter.description())  # I count instances...
print(c1.id, c2.id, c3.id)  # 1 2 3
```

- Class attributes are shared across ALL instances.
- @classmethod — takes cls (the class) as first arg, not self.
- @staticmethod — no self or cls; just a regular function in the class namespace.
- Use @classmethod for factory methods or class-level state.
- Use @staticmethod for utility functions related to the class.

OOP

Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):         # Dog inherits Animal
    def speak(self):
        return f"{self.name} barks!"

class Cat(Animal):         # Cat inherits Animal
    def speak(self):
        return f"{self.name} meows!"

class Puppy(Dog):          # Puppy inherits Dog
    def __init__(self, name):
        super().__init__(name)   # call parent
        self.playful = True

animals = [Dog("Rex"), Cat("Whiskers"), Puppy("Spot")]
for a in animals:
```

- Inheritance: child class gets all methods/attrs of parent.
- class Child(Parent) — declares inheritance.
- Override parent methods by redefining them in child.
- super() calls the parent class's method.
- Polymorphism: different classes, same method name, different behavior.

OOP

Magic Methods (Dunder Methods)

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):          # print(v)
        return f"Vector({self.x}, {self.y})"

    def __repr__(self):         # developer repr
        return f"Vector(x={self.x}, y={self.y})"

    def __add__(self, other):   # v1 + v2
        return Vector(self.x + other.x, self.y + other.y)

    def __len__(self):          # len(v)
        return 2

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2

print(v1)          # Vector(1, 2)
```

- Dunder (double underscore) methods enable operator overloading.
- __str__: defines print() output.
- __repr__: developer-friendly string representation.
- __add__: enables the + operator.
- __len__: enables len() function.
- Also: __eq__, __lt__, __mul__, __getitem__, etc.

ERROR HANDLING

Try / Except — Error Handling

```
# Basic try-except
try:
    x = int(input("Enter a number: "))
    result = 100 / x
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Can't divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")
else:
    print("No errors occurred!")  # runs if no error
finally:
    print("This always runs!")    # cleanup code

# Raise your own exceptions
def check_age(age):
    if age < 0:
        raise ValueError("Age can't be negative!")
    return age
```

- try: code that might fail.
- except: runs if an error (exception) occurs.
- else: runs only if NO exception occurred.
- finally: ALWAYS runs, even after errors (for cleanup).
- raise creates your own exceptions.
- Catch specific exceptions for better error messages!

File Operations

```
# Write to file
with open("data.txt", "w") as f:
    f.write("Hello, File World!\n")
    f.write("Second line.\n")

# Read entire file
with open("data.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line
with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())

# Append to existing file
with open("data.txt", "a") as f:
    f.write("Appended line!\n")

# Read all lines into a list
with open("data.txt") as f:
    lines = f.readlines()
```

Mode	Meaning
"r"	Read (default)
"w"	Write (overwrite)
"a"	Append
"r+"	Read + Write
"b"	Binary (add to any)

- with open() automatically closes the file for you!
- f.read() — entire file as string.
- f.readlines() — list of lines.
- f.readline() — one line at a time.
- Always prefer with statement over manual

FILE I/O

Working with the File System

```
import os

# Current directory
print(os.getcwd())          # /home/user

# List files in directory
print(os.listdir("."))      # ['file1.txt', ...]

# Check if file/folder exists
print(os.path.exists("data.txt"))  # True/False

# Delete a file
if os.path.exists("old_file.txt"):
    os.remove("old_file.txt")

# Create a directory
os.makedirs("my_folder", exist_ok=True)

# Rename a file
os.rename("old.txt", "new.txt")

# Get file info
```

- The os module provides file system operations.
- os.getcwd() — current working directory.
- os.listdir() — list contents of a folder.
- os.path.exists() — check before deleting/opening.
- os.path.join() — build paths that work on all OSes.
- os.makedirs(exist_ok=True) — create nested folders.

Modules & Imports

```
# Import full module
import math
print(math.pi)         # 3.141592653589793
print(math.sqrt(16))   # 4.0
print(math.ceil(4.3))  # 5

# Import specific functions
from math import sqrt, pi, floor
print(sqrt(25))   # 5.0 (no 'math.' needed)

# Import with alias
import random as rnd
print(rnd.randint(1, 10))  # random 1-10
print(rnd.choice(["a","b","c"]))  # random item

# Import your own module
# mymodule.py → def greet(): ...
# import mymodule
# mymodule.greet()

# Useful standard library modules:
# json, datetime, os, sys, re, collections
```

- Modules are .py files you can import and reuse.
- Python has a huge Standard Library — no install needed.
- import module or from module import function.
- Use as to create an alias: import numpy as np.
- Popular 3rd party: requests, numpy, pandas, flask.
- Install 3rd party modules: pip install module_name

ADVANCED

List Comprehensions

```
# Traditional loop
squares = []
for x in range(10):
    squares.append(x ** 2)

# List comprehension (cleaner!)
squares = [x ** 2 for x in range(10)]
print(squares)  # [0, 1, 4, 9, 16, 25, ...81]

# With condition (filter)
evens = [x for x in range(20) if x % 2 == 0]
print(evens)   # [0, 2, 4, 6, ..., 18]

# Nested comprehension
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [num for row in matrix for num in row]
print(flat)   # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Dictionary comprehension
words = ["hello", "world", "python"]
lengths = {word: len(word) for word in words}
print(lengths)  # {'hello': 5, 'world': 5, 'python': 6}
```

- Comprehensions are a Pythonic way to build collections.
- Syntax: [expression for item in iterable if condition]
- Dict comprehension: {k: v for k, v in ...}
- Set comprehension: {expr for item in iterable}
- More concise than loops, but don't over-complicate!

ADVANCED

Lambda Functions

```
# Regular function
def square(x):
    return x ** 2

# Lambda equivalent
square = lambda x: x ** 2
print(square(5))   # 25

# Lambda with multiple args
add = lambda a, b: a + b
print(add(3, 7))   # 10

# Used with sorted()
students = [("Alice", 90), ("Bob", 85), ("Carol", 92)]
sorted_by_grade = sorted(students, key=lambda s: s[1])
print(sorted_by_grade)
# [('Bob', 85), ('Alice', 90), ('Carol', 92)]

# map() — apply to all items
numbers = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x*2, numbers))
print(doubled)   # [2, 4, 6, 8, 10]
```

- Lambda = anonymous (unnamed) one-line function.
- Syntax: lambda args: expression
- Use when you need a simple function once.
- map(func, iterable) applies function to every item.
- filter(func, iterable) keeps items where func returns True.
- sorted() with key= is the most common lambda use case.

ADVANCED

Generators

```
# Generator function (uses yield)
def count_up(n):
    i = 0
    while i < n:
        yield i    # pause and return i
        i += 1

gen = count_up(5)
print(next(gen))   # 0
print(next(gen))   # 1
print(next(gen))   # 2

# Loop over generator
for num in count_up(5):
    print(num, end=" ")   # 0 1 2 3 4

# Generator expression (like list comp)
even_gen = (x for x in range(100) if x % 2 == 0)
print(next(even_gen))   # 0
print(next(even_gen))   # 2

# Infinite generator!
```

- Generators produce values lazily — one at a time.
- Use yield instead of return — pauses the function.
- Memory efficient: don't create the whole list at once.
- Great for large data streams you can't load in memory.
- Generator expressions: (expr for x in ...) with () not [].

ADVANCED

Decorators

```
# Decorator = function that wraps another function
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Done!")
        return result
    return wrapper

@logger
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
# Calling: greet
# Hello, Alice!
# Done!

# Timing decorator
import time
def timer(func):
    def wrapper(*args, **kwargs):
```

- Decorators wrap a function to add extra behavior.
- @decorator is syntactic sugar for: func = decorator(func).
- Common uses: logging, timing, authentication, caching.
- The wrapper function receives *args, **kwargs to pass-through.
- You can stack multiple decorators on one function.

Bitwise Operators

```
a = 12  # binary: 1100
b = 10  # binary: 1010

print(a & b)    # 8   — AND  (1000)
print(a | b)    # 14  — OR   (1110)
print(a ^ b)    # 6   — XOR  (0110)
print(~a)       # -13 — NOT  (flip all bits)
print(a << 1)   # 24  — left shift  (11000)
print(a >> 1)   # 6   — right shift (110)

# Practical example: check if number is even
def is_even(n):
    return (n & 1) == 0

print(is_even(4))   # True
print(is_even(7))   # False

# Flags using bitwise
READ  = 0b100  # 4
WRITE = 0b010  # 2
EXEC  = 0b001  # 1
```

- Bitwise operators work on binary representations of integers.
- & = AND: 1 only if BOTH bits are 1.
- | = OR: 1 if at least one bit is 1.
- ^ = XOR: 1 if bits are DIFFERENT.
- << shifts bits left (multiply by 2), >> shifts right (divide by 2).
- Used in low-level programming, flags, and performance tricks.

Type Checking & Duck Typing

```
# Check type with type()
print(type(42))           # <class 'int'>
print(type("hello"))      # <class 'str'>
print(type([1,2,3]))      # <class 'list'>

# isinstance() — recommended for checking!
print(isinstance(42, int))         # True
print(isinstance(42, (int, float))) # True
print(isinstance("hi", str))       # True

# isnumeric() on strings
print("42".isnumeric())    # True
print("3.14".isnumeric())  # False
print("abc".isnumeric())   # False

# callable() — check if callable
print(callable(print))     # True
print(callable(42))        # False

# Duck typing: "if it walks like a duck..."
def process(item):
    # works on anything with len()
```

- type() returns the exact class of an object.
- isinstance() checks if an object is an instance of a class.
- Use isinstance() in practice — works with inheritance too.
- Duck typing: don't check type — just use the object!
- 'If it has a .append() method, treat it like a list.'

OOP

Class Memory Optimization with __slots__

```
# Without __slots__ — uses a __dict__
class Normal:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Normal(1, 2)
p.z = 99   # can add any attribute!
print(p.__dict__)   # {'x': 1, 'y': 2, 'z': 99}

# With __slots__ — restricted & memory efficient!
class Optimized:
    __slots__ = ["x", "y"]   # only these allowed

    def __init__(self, x, y):
        self.x = x
        self.y = y

p2 = Optimized(1, 2)
# p2.z = 99   ← AttributeError! Not in __slots__
print(p2.x)   # 1
```

- __slots__ restricts which attributes instances can have.
- Removes the per-instance __dict__, saving significant memory.
- Can be 10-15x more memory efficient for large numbers of objects.
- Cannot add arbitrary attributes after __slots__ is set.
- Best used for data-heavy objects (points, records, nodes).

DATABASES

Python + MongoDB

```
# Install: pip install pymongo
from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")
db = client["my_database"]
collection = db["users"]

# Insert one document
user = {"name": "Alice", "age": 30, "city": "NYC"}
result = collection.insert_one(user)
print(result.inserted_id)

# Insert many
users = [
    {"name": "Bob", "age": 25},
    {"name": "Carol", "age": 35},
]
collection.insert_many(users)

# Find documents
for user in collection.find({"age": {"$gt": 25}}):
```

- MongoDB stores data as JSON-like documents.
- pymongo is the official Python driver for MongoDB.
- insert_one() / insert_many() adds documents.
- find() returns a cursor — iterate with for loop.
- Use $gt, $lt, $in operators for queries.
- Great for flexible schemas and rapid prototyping.

String Formatting — Three Ways

```
name = "Alice"
age = 30
pi = 3.14159

# Method 1: % operator (old style)
print("Hello %s, you are %d" % (name, age))
print("Pi is %.2f" % pi)

# Method 2: .format() method
print("Hello {}, you are {}".format(name, age))
print("Hello {name}, age {age}".format(name=name, age=age))
print("Pi: {:.3f}".format(pi))    # Pi: 3.142

# Method 3: f-strings (recommended, Python 3.6+)
print(f"Hello {name}, you are {age}")
print(f"Pi rounded: {pi:.2f}")    # 3.14
print(f"Calc: {2 ** 10}")         # 1024
print(f"{name!u}")                # ALICE (uppercase)
print(f"{age:05d}")               # 00030 (zero pad)
print(f"{'centered':^20}")        #       centered
print(f"{1234567:,}")             # 1,234,567
```

- f-strings are the modern, preferred way (Python 3.6+).
- Supports expressions inside {}: f'{2+2}' → '4'.
- .2f → 2 decimal places for floats.
- :, → adds comma separators to numbers.
- :>10, :<10, :^10 → right, left, center alignment.
- :05d → zero-padded integer.

STDLIB

Python Timers & Time Module

```
import time
import threading

# Measure elapsed time
start = time.time()
# ... some code ...
time.sleep(1)   # pause for 1 second
elapsed = time.time() - start
print(f"Elapsed: {elapsed:.2f}s")

# Monotonic clock (for timing, won't go backward)
start = time.monotonic()
time.sleep(0.5)
print(f"Monotonic: {time.monotonic() - start:.2f}s")

# Schedule a function to run after delay
def ring_alarm():
    print("ALARM! Wake up!")

timer = threading.Timer(3.0, ring_alarm)
timer.start()   # rings after 3 seconds
```

- time.time() returns current time as seconds since epoch.
- time.sleep(n) pauses execution for n seconds.
- time.monotonic() is better for measuring intervals.
- threading.Timer(delay, func) schedules a delayed function call.
- divmod(a, b) returns quotient and remainder as a tuple.

You've Covered Python Grammar!

- Variables & Data Types
- Control Flow (if/else)
- Lists, Tuples, Dicts, Sets
- Error Handling
- Strings & Formatting
- For & While Loops
- Slicing & Iterators
- File I/O
- Operators
- Functions
- OOP & Classes
- Advanced: Lambda, Generators, Decorators

Next Steps: Practice on HackerRank • Build projects • Explore pandas, Flask, Django • Keep coding!